MODULE 2

STACKS

Q1) What is a stack? What are the various operations that can be performed on stack?

A Stack is a special type of data structure (an ordered collection of items) in which insertions (also called pushes and adds) and deletions (also called pops and removes) are made at one end called the top. Using this approach, the Last element Inserted is the first element to be deleted Out, and hence, stack is also called Last In First Out (LIFO) data structure. The Stack $S = (a_0, a_1, \ldots a_{n-1})$ is pictorically represented as follows:

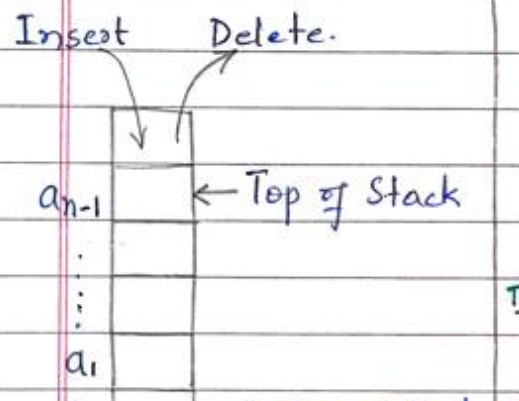


figure: Pictorial representation of Stack

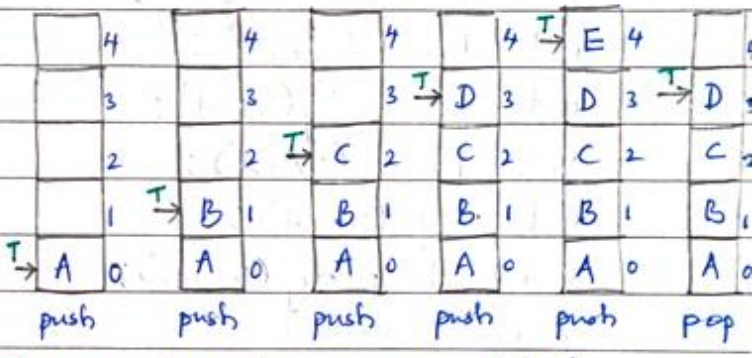


figure: Inserting and deleting elements in a Stack.

The various operations that can be performed on stacks are:

Insert: An element is inserted from top end. Insertion operation is called push operation.

Delete: An element is deleted from top end only. Deletion operation is called pop operation.

Overflow: Check whether the stack is full or not.

Underflow: Check whether the stack is empty or not.

Q2) Explain the array representation of Stack?

In the computer's memory, stacks can be represented as a linear array. Every stack has a variable called TOP associated with it, which is used to store the address of the topmost element of the stack. It is this position where the element will be added to or deleted from. There is another variable called MAX, which is used to store the maximum number of elements that the stack can hold. If TOP = NULL, then it indicates that the stack is empty and if TOP = MAX–1, then the stack is full.

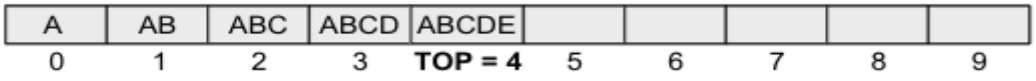


Figure Stack

The stack in Figure above shows that TOP=4, so, insertions and deletions will be done at this position. In the above stack, 5 more elements can be inserted, because the size of the stack is 10.

Q3) Explain the different operations that can be performed on a stack?

Push Operation:

The push operation is used to insert an element into the stack. The new element is added at the topmost position of the stack. However, before inserting the value, we must first check if TOP=MAX–1, because if that is the case, then the stack is full and no more insertions can be done. If an attempt is made to insert a value in a stack that is already full, an OVERFLOW message is printed. Consider the stack given in Figure 1.

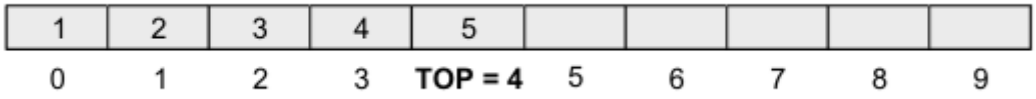


Figure 1: Stack

To insert an element with value 6, we first check if TOP=MAX–1. If the condition is false, then we increment the value of TOP and store the new element at the position given by stack[TOP]. Thus, the updated stack becomes as shown in Figure 2.

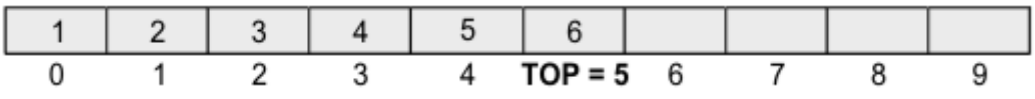


Figure 2: Stack after insertion

Figure 3 shows the algorithm to insert an element in a stack. In Step 1, we first check for the OVERFLOW condition. In Step 2, TOP is incremented so that it points to the next location in the array. In Step 3, the value is stored in the stack at the location pointed by TOP.

```
Step 1: IF TOP = MAX-1
            PRINT "OVERFLOW"
            Goto Step 4
        [END OF IF]
Step 2: SET TOP = TOP + 1
Step 3: SET STACK[TOP] = VALUE
Step 4: END
```

Figure 3: Algorithm to insert an element in a stack

Pop Operation:

The pop operation is used to delete the topmost element from the stack. However, before deleting the value, we must first check if TOP=NULL because if that is the case, then it means the stack is empty and no more deletions can be done. If an attempt is made to delete a value from a stack that is already empty, an UNDERFLOW message is printed.

Consider the stack given in Figure 1. To delete the topmost element, we first check if TOP=NULL. If the condition is false, then we decrement the value pointed by TOP. Thus, the updated stack becomes as shown in Figure 3.

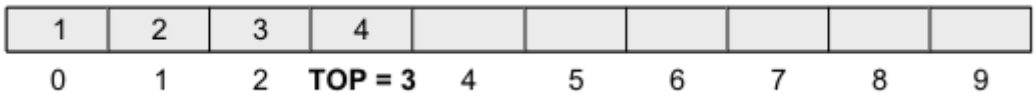


Figure 3: Stack after deletion.

Figure 4 shows the algorithm to delete an element from a stack. In Step 1, we first check for the UNDERFLOW condition. In Step 2, the value of the location in the stack pointed by TOP is stored in VAL. In Step 3, TOP is decremented.

```
Step 1: IF TOP = NULL
            PRINT "UNDERFLOW"
            Goto Step 4
        [END OF IF]
Step 2: SET VAL = STACK[TOP]
Step 3: SET TOP = TOP - 1
Step 4: END
```

Figure 4: Algorithm to delete an element from a stack

Peek Operation:

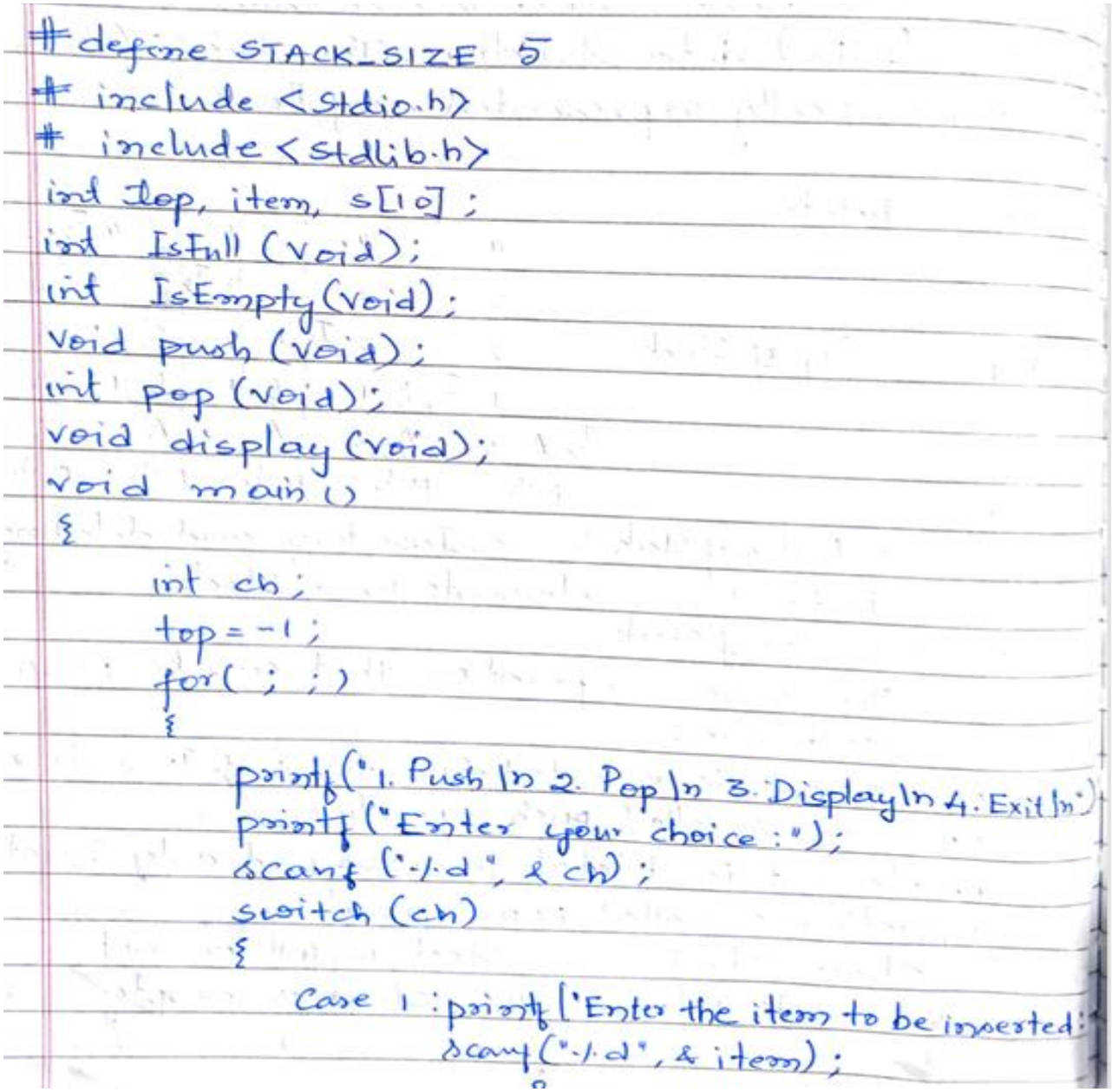
Peek is an operation that returns the value of the topmost element of the stack without deleting it from the stack.

However, the Peek operation first checks if the stack is empty, i.e., if TOP = NULL, then an appropriate message is printed, else the value is returned. Consider the stack given in Figure 1, the Peek operation will return 5, as it is the value of the topmost element of the stack. The algorithm for Peek operation is given in Figure 5.

```
Step 1: IF TOP = NULL
            PRINT "STACK IS EMPTY"
            Goto Step 3
Step 2: RETURN STACK[TOP]
Step 3: END
```

Figure 5: Algorithm for Peek operation

Q4) Give the C implementation of push and pop function. Include check to empty and full condition of stack.



```
#define STACK_SIZE 5
#include <stdio.h>
#include <stdlib.h>
int top, item, s[10];
int IsFull (void);
int IsEmpty(void);
void push (void);
int pop (void);
void display (void);
void main ()
{
    int ch;
    top = -1;
    for( ; ; )
    {
        printf("1. Push \n 2. Pop \n 3. Display \n 4. Exit \n")
        printf ("Enter your choice :");
        scanf ("%d", &ch);
        switch (ch)
        {
            case 1 :printf ("Enter the item to be inserted:
                    scanf("%d", &item);
```

```
                push();
                break;
            case 2: item = pop();
                if (item == -1)
                    printf("Stack is Empty\n");
                else
                    printf("Item deleted = %d\n", item);
                break;
            case 3: display();
                break;
            default: exit(0);
        }
    }
}
```

```
int IsFull(void)
{
    return top == STACK_SIZE - 1;
}
int IsEmpty(void)
{
    return top == -1;
}
```

```
void push(void)
{
    if (IsFull())
    {
        printf("Stack Overflow\n");
        return;
    }
    s[++top] = item;
}
```

```
int pop(void)
{
    if (IsEmpty())
        return -1;
    return s[top--];
}
```

```
void display(void)
{
    int i;
    if (top == -1)
    {
        printf("Stack is empty\n");
        return;
    }
    printf("Contents of the Stack\n");
    for(i=0; i<=top; i++)
    {
        printf("%d\n", s[i]);
    }
}
```

Q5) Write a program in c to check whether the given string is palindrome or not.

```
#define STACK_SIZE 25
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int top;
char stack[25];
char str[25];
int count, slen;
```

```
void push(char);
char pop(void);
void palindrome(void);
void main()
{
    int i, top=-1, count=0;
    printf("Enter any string:");
    gets(str);
    slen = strlen(str);
    for(i=0; i<slen; i++)
    {
        push(str[i]);
    }
    palindrome();
    if(count == slen)
        printf("String is Palindrome\n");
    else
        printf("String is not a Palindrome\n");
}
```

```
void push (char ch)
{
    if (top == STACK-SIZE - 1)
    {
        printf("stack is Overflow\n");
        return;
    }
    Stack [++top] = ch;
}
```

```
char pop (void)
{
    if (top == -1) { printf("Stack is Underflow\n");
        return '\0';
    }
    else
        return stack[top--];
}
void palindrome(void)
{
    int i; char ch;
    for (i=0; i<slen; i++)
    {
        ch = pop();
        if (ch == str[i])
            count++;
    }
}
```

Q6) What are the applications of stack?

The various applications of stack are as follows:

1. Conversion of expressions from infix expression into postfix expression.
2. Conversion of expressions from infix expression into prefix expression.
3. Evaluation of postfix expression.
4. Evaluation of prefix expression.
5. Recursion.
6. To check whether the given string is palindrome or not.
7. To check whether the given expression is valid or not (Parenthesis checker).
8. Reversing a list.

Note: **Parenthesis-free Prefix Notation** is known as **Polish Notation** and **Postfix Notation** is known as **Reverse Polish Notation**.

Q7) Write a program in C to reverse a list of given numbers.

```
#include <stdio.h>
#include <conio.h>
int stk[10];
int top=–1;
int pop();
void push(int);
main()
{
        int val, n, i,
        arr[10];
        clrscr();
        printf("\n Enter the number of elements in the array : ");
        scanf("%d", &n);
        printf("\n Enter the elements of the array : ");
        for(i=0;i<n;i++)
                scanf("%d", &arr[i]);
        for(i=0;i<n;i++)
                push(arr[i]);
        for(i=0;i<n;i++)
        {
                val = pop();
                arr[i] = val;
        }
        printf("\n The reversed array is : ");
        for(i=0;i<n;i++)
                printf("\n %d", arr[i]);
        getche"();
}
void push(int val)
{
        stk[++top] = val;
}
int pop()
{
        return(stk[top—]);
}
```

Output:

Enter the number of elements in the array : 5
Enter the elements of the array : 1 2 3 4 5
The reversed array is : 5 4 3 2 1

Q8) Write a program in C to check nesting of parenthesis using a stack (Implementation of Parenthesis checker).

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
#define MAX 10
int top = –1;
int stk[MAX];
void push(char);
char pop();
main()
{
        char exp[MAX],temp;
        int i, flag=1;
        clrscr();
        printf("Enter an expression : ");
        gets(exp);
        for(i=0;i<strlen(exp);i++)
        {
                if(exp[i]=='(' || exp[i]=='{' || exp[i]=='[')
                        push(exp[i]);
                if(exp[i]==')' || exp[i]=='}' || exp[i]==']')
                if(top == –1)
                        flag=0;
                else
                {
                        temp=pop();
                        if(exp[i]==')' && (temp=='{' || temp=='['))
                                flag=0;
                        if(exp[i]=='}' && (temp=='(' || temp=='['))
                                flag=0;
                        if(exp[i]==']' && (temp=='(' || temp=='{'))
                                flag=0;
                }
        }
        if(top>=0)
                flag=0;
        if(flag==1)
                printf("\n Valid expression");
        else
                printf("\n Invalid expression");
}
```

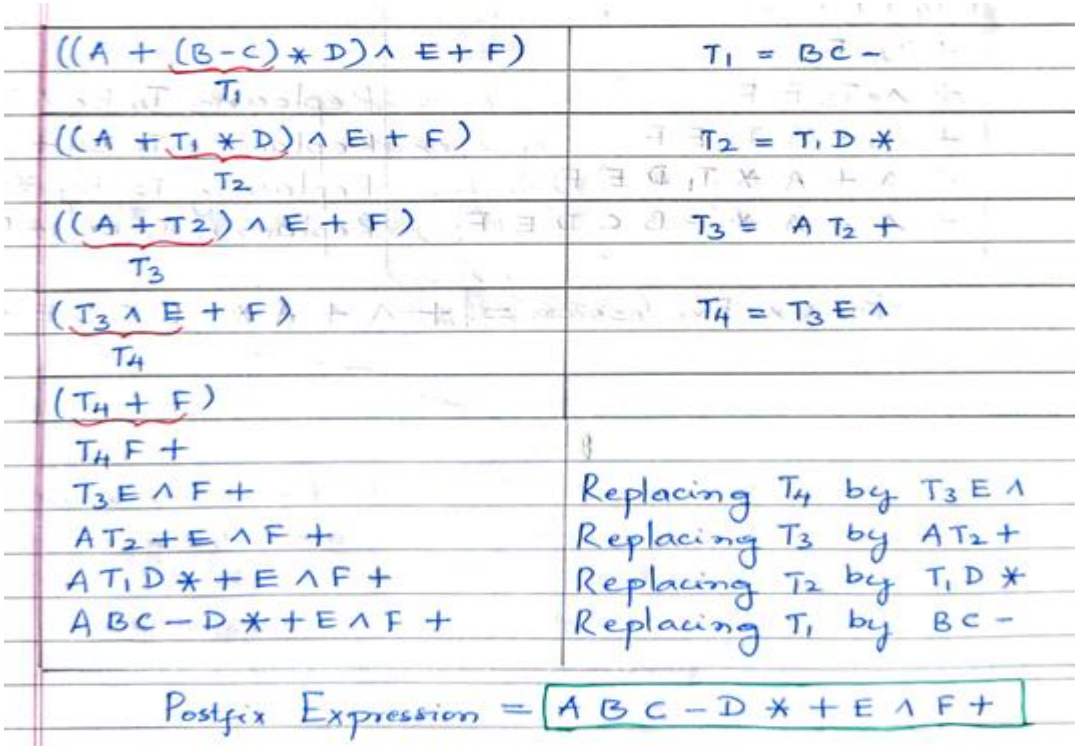
```
void push(char c)
{
        if(top == (MAX–1))
                printf("Stack Overflow\n");
        else
        {
                top=top+1;
                stk[top] = c;
        }
}
char pop()
{
        if(top == –1)
                printf("\n Stack Underflow");
        else
                return(stk[top—]);
}
```

Output:

Enter an expression : (A + (B – C))
Valid Expression

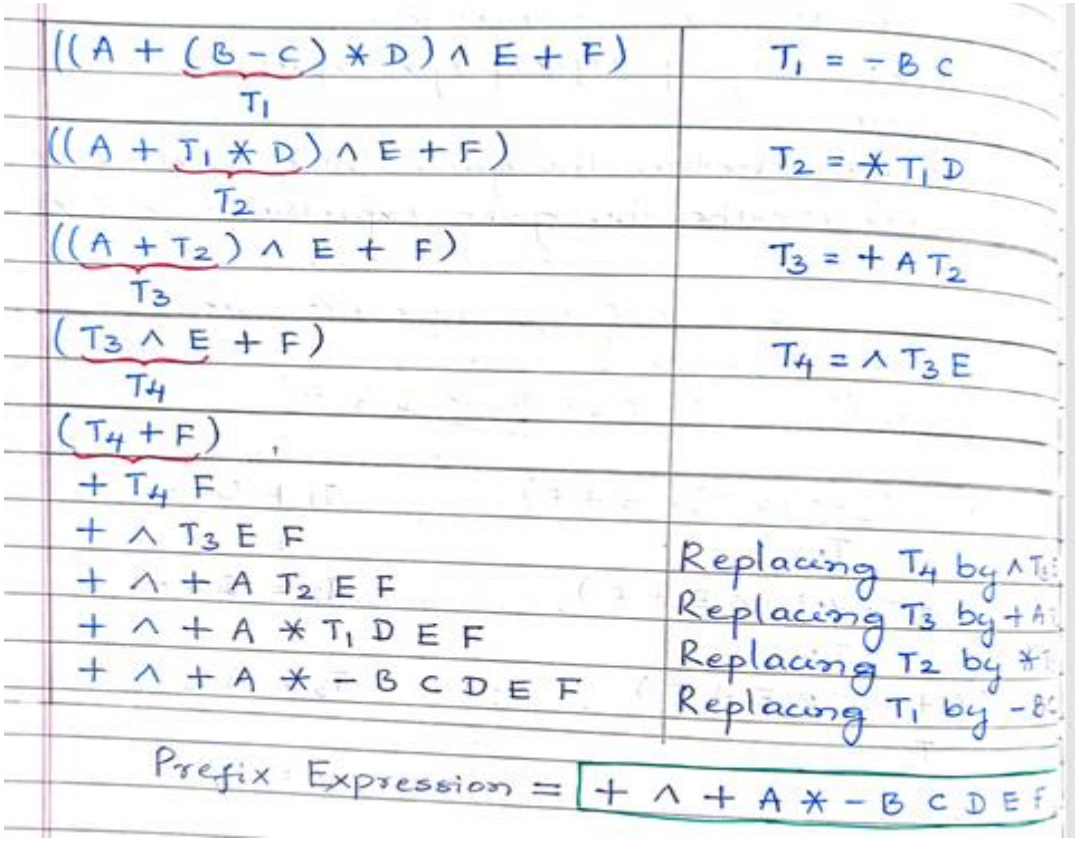
Q9) Convert the following infix expression into postfix expression.

((A + (B – C) * D) ^ E + F)



Q10) Convert the following infix expression into prefix expression.

((A + (B – C) * D) ^ E + F)



Q11) Write a function in C to convert the given infix expression to its equivalent postfix expression.

```
typedef enum {lparen, rparen, plus, minus, times, divide,
              mod, eos, operand} precedence;
/* isp and icp arrays -- index is value of precedence
   lparen, rparen, plus, minus, times, divide, mod, eos */
int isp[] = {0, 19, 12, 12, 13, 13, 13, 14, 0};
int icp[] = {20, 19, 12, 12, 13, 13, 13, 14, 0};
int top = 0;
char stack[20];
```

```
/* Function to get a token from the input String */
precedence getToken(char *symbol, int *n)
{
    *symbol = expr[(*n)++];
    switch (*symbol)
    {
        case '(' : return lparen;
        case ')' : return rparen;
        case '+' : return plus;
        case '-' : return minus;
        case '/' : return divide;
        case '*' : return times;
        case '%' : return mod;
        case ' ' : return eos;
        default : return operand;
    }
}
```

```
/* Function to Convert from infix to postfix */
void postfix(void)
{
    char symbol;
    precedence token;
    int n = 0;
    int top = 0;
    stack[0] = eos;
    for (token = getToken(&symbol, &n); token != eos;
                token = getToken(&symbol, &n))
    {
        if (token == operand)
            printf("%c", symbol);
        else if (token == rparen)
        {
```

```
            while( Stack[top] != lparen)
                printToken(pop());
            pop(); /* discard the left parenthesis */
        }
        else
        {
            while(isp[stack[top]] >= icp[token])
                printToken(pop());
            push(token);
        }
    }
    while ( (token = pop()) != eos)
        printToken(token);
    printf("\n");
}
```

Example 1 : Translation of a*(b+c)*d to postfix

Token	Stack [0]	[1]	[2]	[3]	Top	Output
	eos				0	
a	eos				0	a
*	eos	*			1	a
(	eos	*	(		2	a
b	eos	*	(		2	ab
+	eos	*	(	+	3	ab
c	eos	*	(	+	3	abc
)	eos	*			1	abc+
*	eos	*			1	abc+*
d	eos	*			1	abc+*d
eos	eos				0	abc+*d*

Example 2 : Translation of a+b*c to postfix

Token	Stack [0]	[1]	[2]	[3]	Top	Output
	eos				0	
a	eos				0	a
+	eos	+			1	a
b	eos	+			1	ab
*	eos	+	*		2	ab
c	eos	+	*		2	abc
eos	eos				0	abc*+

Q12) Write a function in C to evaluate postfix expression.

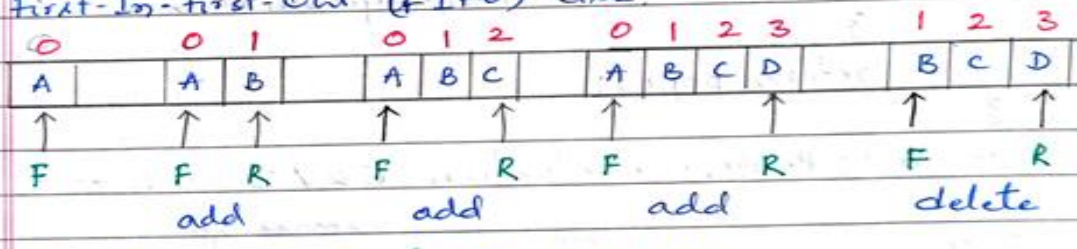
```
/* Function to evaluate a postfix expression */
int eval (void)
{
    precedence token;
    char symbol;
    int op1, op2;
    int n = 0;
    int top = -1;
    token = getToken (&Symbol, &n);
    while (token != eos)
    {
        if (token == operand)
            push (Symbol - '0')
        else
        {
            op2 = pop ();
            op1 = pop ();
            switch (token)
            {
                case plus : push (op1 + op2);
                            break;
                Case minus : push (op1 - op2);
                            break;
                Case times : push (op1 * op2);
                            break;
                Case divide : push (op1 / op2);
                            break;
                Case mod : push (op1 % op2);
                            break;
            }
        }
        token = getToken (&Symbol, &n);
    }
    return pop();
}
```

Example 1: Postfix evaluation 6 2 / 3 – 4 2 * +

Token	Stack [0]	[1]	[2]	[3]	Top	Output
6	6				0	
2	6	2			1	
/	3				0	
3	3	3			1	
–	0				0	
4	0	4			1	
2	0	4	2		2	
*	0	8			1	
+	8				0	
eos						8

Q13) What is Queue? What are the operations that can be performed on a queue?

A queue is a special type of data structure (an ordered collection of items), in which insertions (also called additions, puts and pushes) and deletions (also called removals and pops) take place at different ends. The end at which new elements are added is called the Rear End, and that from which old elements are deleted is called the Front End. Since, the first element inserted into a queue is the first element removed. Hence, queue are also known as First-In-First-Out (FIFO) lists.



F = Queue Front, R = Queue Rear

Figure: Inserting and deleting elements in a queue

The various operations that can be perfmed on queue are

Insert: An element is inserted from Rear end.

Delete: An element is deleted from Front end.

Overflow: If Queue is full, and if we try to insert an item, an overflow condition occurs.

Underflow: If Queue is empty, and if we try to delete an item, an underflow condition occurs.

Different types of Queues are

1. Linear Queue (Ordinary Queue)
2. Circular Queue
3. Double Ended Queue (Dequeue)
4. Priority Queue

Q14) Explain array representation of queues.

Queues can be easily represented using linear arrays. Every queue has front and rear variables that point to the position from where deletions and insertions can be done, respectively.
The array representation of a queue is shown in Figure 1.

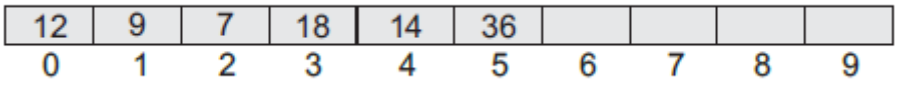


Figure 1: Queue

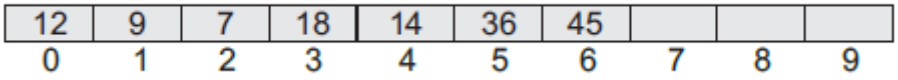


Figure 2: Queue after insertion of a new element

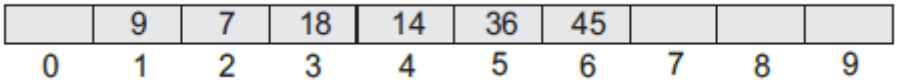


Figure 3: Queue after deletion of an element

Operations on Queues:

In Figure 1, FRONT = 0 and REAR = 5. If we want to add another element with value 45, then REAR should be incremented by 1 and the value can be stored at the position pointed by REAR. The representation of queue after addition is as shown in Figure2. Here, FRONT = 0 and REAR = 6. Whenever a new element has to be added, we have to repeat the same procedure.

If we want to delete an element from the queue, then the value of FRONT will be incremented. Deletions are done from only this end of the queue. The representation of queue after deletion is as shown in Figure 3. Here, FRONT = 1 and REAR = 6.

Before inserting an element in a queue, we must check for overflow conditions. An overflow will occur when we try to insert an element into a queue that is already full. When REAR = MAX – 1, where MAX is the size of the queue, we have an overflow condition.

Similarly, before deleting an element from a queue, we must check for underflow conditions. An underflow condition occurs when we try to delete an element from a queue that is already empty. If FRONT = –1 and REAR = –1, it means there is no element in the queue.

Figure 4 and Figure 5 shows the algorithms to insert and delete an element from a queue.

```
Step 1: IF REAR = MAX-1
            Write OVERFLOW
            Goto step 4
        [END OF IF]
Step 2: IF FRONT = -1 and REAR = -1
            SET FRONT = REAR = 0
        ELSE
            SET REAR = REAR + 1
        [END OF IF]
Step 3: SET QUEUE[REAR] = NUM
Step 4: EXIT
```

Figure 4: Algorithm to insert an element in a Queue

```
Step 1: IF FRONT = -1 OR FRONT > REAR
            Write UNDERFLOW
        ELSE
            SET VAL = QUEUE[FRONT]
            SET FRONT = FRONT + 1
        [END OF IF]
Step 2: EXIT
```

Figure 5: Algorithm to delete an element from a Queue

Q15) Write a function in C to add, delete and display the elements from queues?

```
#dgine MAX_QUEUE_SIZE 100
typedef struct
{
    int key;
}element;
element queue[MAX_QUEUE_SIZE];
int rear = -1;
int front = -1;
Boolean IsEmptyQ(void);
Boolean IsFullQ(void);
void addQ(element item);
int deleteQ(void);
void display(void);

Boolean IsEmptyQ(void)
{
    if (front >= rear)
        return TRUE;
    else
        return FALSE;
}
Boolean IsFullQ(void)
{
    if (rear == MAX_QUEUE_SIZE - 1)
        return TRUE;
    else
        return FALSE;
}
```

```
void addQ(element item)
{
    if(IsFullQ())
    {
        printf("Queue Overflow\n");
        return;
    }
    queue[++rear] = item;
}
int deleteQ(void)
{
    if(IsEmptyQ())
        return -1;
    return queue[++front];
}
```

```
void display(void)
{
    int i;
    if (front > rear)
    {
        printf("Queue is empty\n");
        return;
    }
    printf("Contents of the queue are\n");
    for(i = front; i<=rear; i++)
    {
        printf("%d\n", queue[i]);
    }
}
```

Example:

Front	Rear	Queue Q[0]	Q[1]	Q[2]	Q[3]	Comments
-1	-1					Queue is empty
-1	0	J_1				Job1 is added
-1	1	J_1	J_2			Job2 is added
-1	2	J_1	J_2	J_3		Job3 is added
0	2		J_2	J_3		Job1 is deleted
1	2			J_3		Job2 is deleted

Figure: Insertion and Deletion from a Sequential Queue

Q16) What is circular queue? Write a program in C to implement insert, delete and display operations on a circular queue using array.

In a circular queue, the elements of a given queue can be stored efficiently in an array so as to 'wrap around' so that end of the queue is followed by the front of queue.

The pictorial representation of a circular queue and its equivalent representation using an array is shown below:

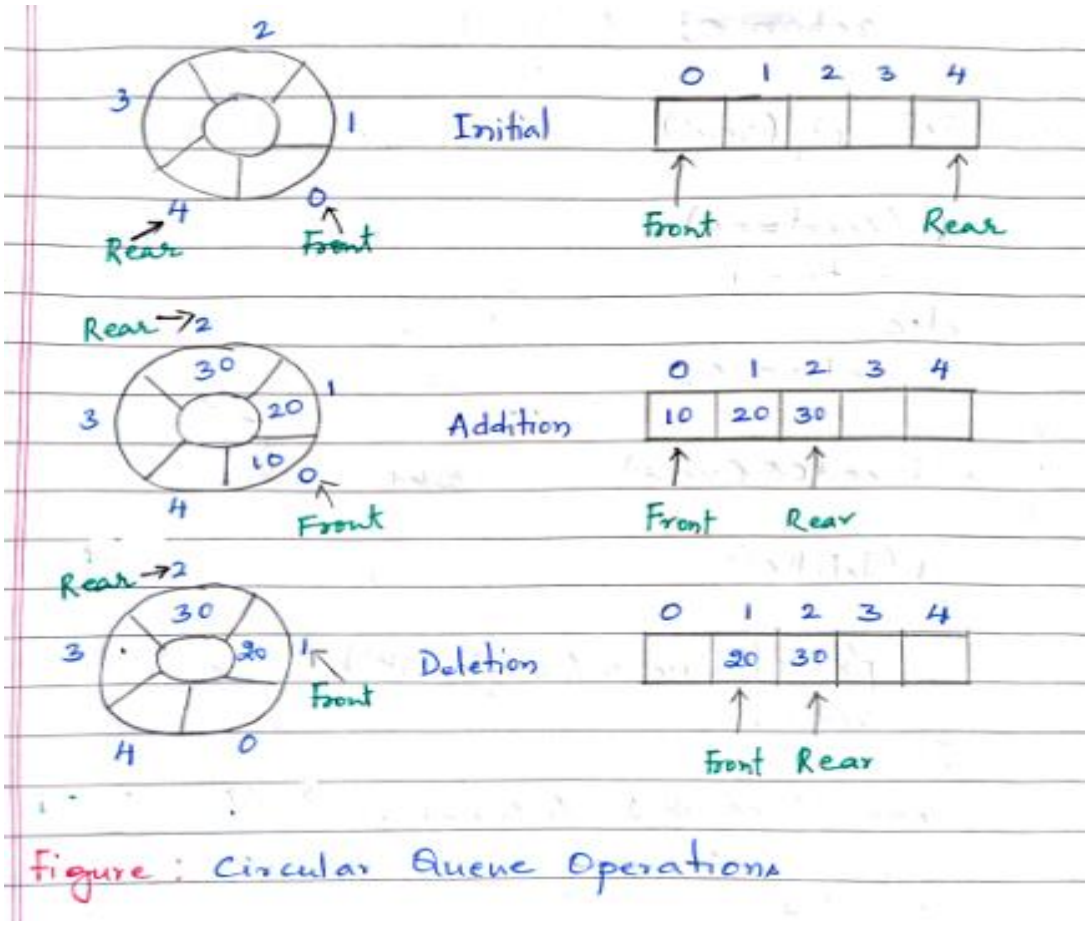


Figure : Circular Queue Operations

```
#define QSIZE 5
int q[QSIZE];
int front = 0;
int rear = -1;
int count = 0;
int IsFull(void)
{
    if (count == QSIZE)
        return 1;
    else
        return 0;
}
int IsEmpty(void)
{
    if (count == 0)
        return 1;
    else
        return 0;
}
```

```
void InsertCQ(void)
{
    if(IsFull())
    {
        printf("Queue Overflow\n");
        return;
    }
    rear = (rear + 1) % QSIZE;
    q[rear] = item;
    Count++;
}
```

```
int DeleteCQ()
{
    int item;
    if(IsEmpty())
        return -1;
    item = q[front];
    front = (front + 1) % QSIZE;
    Count--;
    return item;
}
void display()
{
    int i;
    if(IsEmpty())
    {
        printf("Queue is Empty\n");
        return;
    }
    printf("Contents of Queue are \n");
    for(i=1; i<=Count; i++)
    {
        printf("%d\n", q[front]);
        front = (front+1) % QSIZE;
    }
}
```

Q17) Write an algorithm to insert an element in a circular queue and delete an element from a circular queue.

```
Step 1: IF FRONT = 0 and Rear = MAX - 1
            Write "OVERFLOW"
            Goto step 4
        [End OF IF]
Step 2: IF FRONT = -1 and REAR = -1
            SET FRONT = REAR = 0
        ELSE IF REAR = MAX - 1 and FRONT != 0
            SET REAR = 0
        ELSE
            SET REAR = REAR + 1
        [END OF IF]
Step 3: SET QUEUE[REAR] = VAL
Step 4: EXIT
```

Figure 1: Algorithm to insert an element in a circular queue.

```
Step 1: IF FRONT = -1
            Write "UNDERFLOW"
            Goto Step 4
        [END of IF]
Step 2: SET VAL = QUEUE[FRONT]
Step 3: IF FRONT = REAR
            SET FRONT = REAR = -1
        ELSE
            IF FRONT = MAX -1
                  SET FRONT = 0
            ELSE
                  SET FRONT = FRONT + 1
            [END of IF]
        [END OF IF]
Step 4: EXIT
```

Figure 2: Algorithm to delete an element from a circular queue.

Q18) What is double ended queue (or dequeuer)? What are the variants of double-ended queue? What are the operations that can be performed on dequeue?

A deque (pronounced as 'deck' or 'dequeue') is a list in which the elements can be inserted or deleted at either end. It is also known as a head-tail linked list because elements can be added to or removed from either the front (head) or the back (tail) end. However, no element can be added and deleted from the middle.

In the computer's memory, a deque is implemented using either a circular array or a circular doubly linked list. In a deque, two pointers are maintained, LEFT and RIGHT, which point to either end of the deque. The elements in a deque extend from the LEFT end to the RIGHT end and since it is circular, Dequeue[N–1] is followed by Dequeue[0].

Consider the deques shown in Figure 1.
There are two variants of a double-ended queue. They include

1) Input restricted dequeuer
2) Output restricted dequeuer

Input restricted deque: In this dequeue, insertions can be done only at one of the ends, while deletions can be done from both ends.

Output restricted deque: In this dequeue, deletions can be done only at one of the ends, while insertions can be done on both ends.

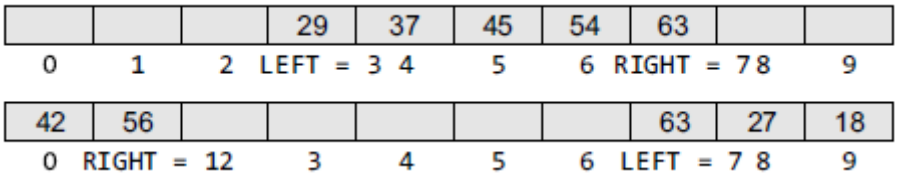
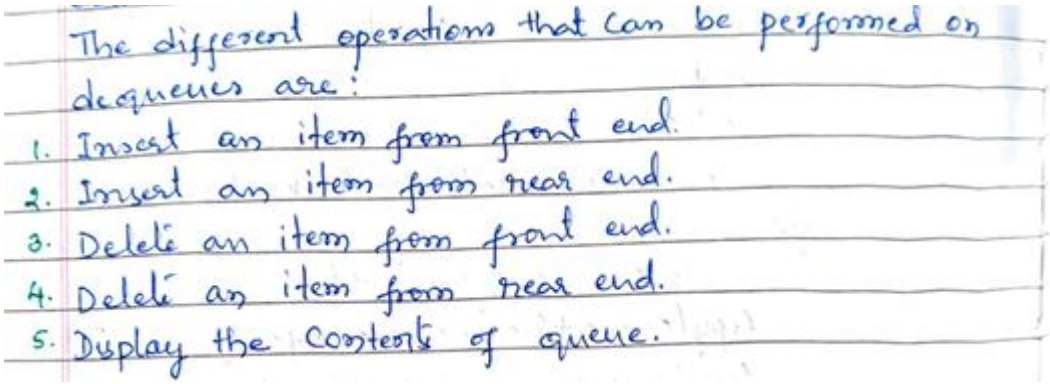
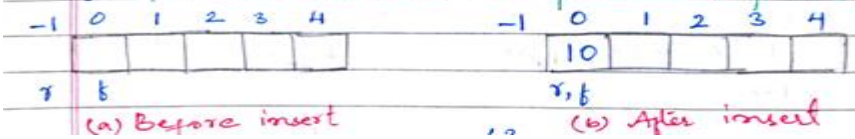


Figure 1: Double-ended queues



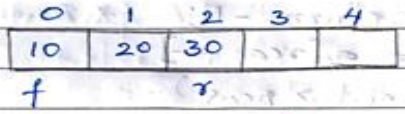
```
/* Function to insert an item from front end */
void Insert_Front (void)
{       /* Case 1 : Insert when queue is empty */
    if (front == 0 && rear == -1)
    {
        q[++rear] = item;
        return;
    }
    if (front != 0) /* Case 2 : Insert when items are present */
    {
        q[--front] = item;
        return;
    }
    printf("Front insertion is not possible \n");
}   /* Case 3 : Insertion not possible at front end */
```



(a) Before insert (b) After insert



(c) After deleting 2 items (d) After inserting 20

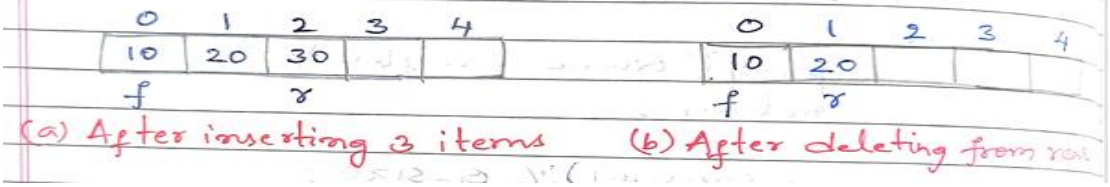


(e) Queue after inserting 10,20,30

```
/* Insert an item from rear end */
void Insert_Rear()
{
    if (rear == Q_SIZE - 1)
    {
        printf("Queue Overflow \n");
        return;
    }
    rear = rear + 1;
    q[rear] = item;
}

/* Function to delete an item from front end */
int Delete_Front()
{
    if (front > rear)
        return -1;
    return q[front++];
}
```

```
/* Function to delete an item from the rear end */
int Delete_Rear()
{
    int item;
    if (front > rear)
        return -1; /* Queue is empty */
    item = q[rear--];
    if (front > rear)
    {
        front = 0; /* Reset to initial state
        rear = -1;    of empty Queue */
    }
    return item;
}
```



(a) After inserting 3 items (b) After deleting from rear

```
/* Function to display contents of queue */
void Display()
{
    int i;
    if (front > rear) /* If Queue is empty */
    {
        printf("Queue is empty\n");
        return;
    }
    /* Display contents of Queue */
    printf("Contents of the queue are\n");
    for(i = front; i <= rear; i++)
        printf("%d\n", q[i]);
}
```

Q19) Define priority queues? What are the general rules to be followed while processing the elements of a priority queue? Explain the different ways in which priority can be implemented?

A priority queue is a data structure in which each element is assigned a priority. The priority of the element will be used to determine the order in which the elements will be processed.

The general rules of processing the elements of a priority queue are

1. An element with higher priority is processed before an element with a lower priority.
2. Two elements with the same priority are processed on a first-come-first-served (FCFS) basis.

There are two ways to implement a priority queue. They are

1. Linked representation of a priority queue.
2. Array representation of a priority queue.

Linked representation of a priority queue:

In the computer memory, a priority queue can be represented using arrays or linked lists. When a priority queue is implemented using a linked list, then every node of the list will have three parts:

(a) The information or data part,
(b) The priority number of the element, and
(c) The address of the next element.

If we are using a sorted linked list, then the element with the higher priority will precede the element with the lower priority.
Consider the priority queue shown in Figure 1.

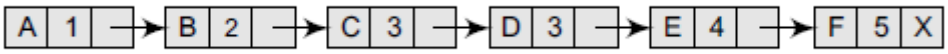


Figure 1: Priority Queue

Lower priority number means higher priority. For example, if there are two elements A and B, where A has a priority number 1 and B has a priority number 5, then A will be processed before B as it has higher priority than B.

Insertion When a new element has to be inserted in a priority queue, we have to traverse the entire list until we find a node that has a priority lower than that of the new element. The new node is inserted before the node with the lower priority. However, if there exists an element that has the same priority as the new element, the new element is inserted after that element.
For example, consider the priority queue shown in Figure 2.



Figure 2: Priority Queue

If we have to insert a new element with data = F and priority number = 4, then the element will be inserted before D that has priority number 5, which is lower priority than that of the new element. So, the priority queue now becomes as shown in Figure 3.

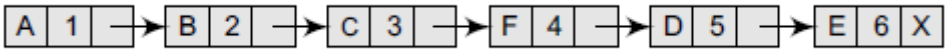


Figure 3: Priority queue after insertion of a new node

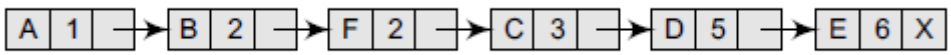


Figure 4: Priority queue after insertion of a new node

Deletion Deletion is a very simple process in this case. The first node of the list will be deleted and the data of that node will be processed first.

Array representation of a priority queue:

When arrays are used to implement a priority queue, then a separate queue for each priority number is maintained. Each of these queues will be implemented using circular arrays or circular queues. Every individual queue will have its own FRONT and REAR pointers.

Two-dimensional array is used to implement priority queue, where each queue will be allocated the same amount of space. Two-dimensional representation of a priority queue given below.

Given the front and rear values of each queue, the two-dimensional matrix can be formed as shown in Figure 5. FRONT[K] and REAR[K] contain the front and rear values of row K, where K is the priority number.

FRONT	REAR
3	3
1	3
4	5
4	1

	1	2	3	4	5
1			A		
2	B	C	D		
3				E	F
4	I			G	H

Figure 5: Priority queue matrix

FRONT	REAR
3	3
1	3
4	1
4	1

	1	2	3	4	5
1			A		
2	B	C	D		
3	R			E	F
4	I			G	H

Figure 6: Priority queue matrix after insertion of a new element

Insertion To insert a new element with priority K in the priority queue, add the element at the rear end of row K, where K is the row number as well as the priority number of that element.

For example, if we have to insert an element R with priority number 3, then the priority queue will be given as shown in Figure 2.

Deletion To delete an element, we find the first nonempty queue and then process the front element of the first non-empty queue. In our priority queue, the first non-empty queue is the one with priority number 1 and the front element is A, so A will be deleted and processed first. In technical terms, find the element with the smallest K, such that FRONT[K] != NULL.

20) Convert the following infix expressions to their postfix equivalents.

(a) A – B + C
Answer: ABC+–

(b) A * B + C / D
Answer: AB*CD/+

(c) (A – B) + C * D / E – C
Answer: AB–CD*EC–/+

(d) (A * B) + (C / D) – (D + E)
Answer: AB*CD/DE+–+

(e) ((A – B) + D / ((E + F) * G))
Answer: AB–DEF+G*/+

(f) (A – 2 * (B + C) / D * E) + F
Answer: A2BC+*D/E*–F+

(g) 14 / 7 * 3 – 4 + 9 / 2
Answer: 14 7 / 3 4 9 2 / + – *

21) Convert the following infix expressions to their prefix equivalents.

(a) A – B + C
Answer: +-ABC

(b) A * B + C / D
Answer: +*AB/CD

(c) (A – B) + C * D / E – F
Answer: -+-AB/*CDEF

(d) (A * B) + (C / D) – (D + E)
Answer: -+*AB/CD+DE

(e) ((A – B) + D / ((E + F) * G))
Answer: +-AB/D*+EFG

(f) (A – 2 * (B + C) / D * E) + F
Answer: +-A*/*2+BCDEF

(g) 14 / 7 * 3 – 4 + 9 / 2
Answer: + - * / 14 7 3 4 / 9 2

22) Convert the following infix expression into its corresponding postfix expression and then evaluate it.

10 + ((7 – 5) + 10)/2
Answer: 10 7 5 – 10 + 2 / +
16

23) Draw the stack structure in each case when the following operations are performed on an empty stack. Stack size = 7.

(a) Add A, B, C, D, E, F
(b) Delete two letters
(c) Add G
(d) Add H
(e) Delete four letters
(f) Add I

24) Draw the queue structure in each case when the following operations are performed on an empty queue.

(a) Add A, B, C, D, E, F
(b) Delete two letters
(c) Add G (d) Add H
(e) Delete four letters (f) Add I

25) Consider the queue given below which has FRONT = 1 and REAR = 5.

	A	B	C	D	E				

Now perform the following operations on the queue:
(a) Add F
(b) Delete two letters
(c) Add G
(d) Add H
(e) Delete four letters
(f) Add I

26) Multiple Choice Questions

1. Stack is a
(a) LIFO (b) FIFO (c) FILO (d) LILO
Answer: (a) LIFO, (c) FILO

2. Which function places an element on the stack?
(a) Pop() (b) Push() (c) Peek() (d) isEmpty()
Answer: (b) Push

3. Disks piled up one above the other represent a
(a) Stack (b) Queue (c) Linked List (d) Array
Answer: (a) Stack

4. Reverse Polish notation is the other name of
(a) Infix expression (b) Prefix expression (c) Postfix expression (d) Algebraic expression
Answer: Postfix expression

5. A line in a grocery store represents a
(a) Stack
(b) Queue
(c) Linked List
(d) Array
Answer: Stack

6. In a queue, insertion is done at
(a) Rear
(b) Front
(c) Back
(d) Top
Answer: Rear

7. The function that deletes values from a queue is called
(a) enqueue
(b) dequeue
(c) pop
(d) peek
Answer: dequeue

8. The function that inserts values from a queue is called
(a) enqueue
(b) dequeue
(c) pop
(d) peek
Answer: enqueue

9. Typical time requirement for operations on queues is
(a) O(1)
(b) O(n)
(c) O(log n)
(d) O(n2)
Answer: O(1)

10. Typical time requirement for operations on stack is
(a) O(1)
(b) O(n)
(c) O(log n)
(d) O(n2)
Answer: O(1)

11. The circular queue will be full only when
(a) FRONT = MAX –1 and REAR = Max –1
(b) FRONT = 0 and REAR = Max –1
(c) FRONT = MAX –1 and REAR = 0
(d) FRONT = 0 and REAR = 0
Answer: FRONT = MAX –1 and REAR = 0

27) True or False

1. Pop() is used to add an element on the top of the stack.
Answer: FALSE

2. Postfix operation does not follow the rules of operator precedence.
Answer: TRUE

3. The peek operation displays the top most value and deletes it from the stack.
Answer: TRUE

4. In a stack, the element that was inserted last is the first one to be taken out.
Answer: TRUE

5. Underflow occurs when TOP = MAX-1.
Answer: FALSE

6. A queue stores elements in a manner such that the first element is at the beginning of the list and the last element is at the end of the list.
Answer: TRUE

7. Elements in a priority queue are processed sequentially.
Answer: FALSE

8. In a priority queue, two elements with the same priority are processed on a FCFS basis.
Answer: TRUE

9. Output-restricted deque allows deletions to be done only at one end of the dequeue, while insertions can be done at both the ends.
Answer: TRUE

10. If front=MAX – 1 and rear= 0, then the circular queue is full.
Answer: TRUE

28) Fill in the Blanks

1. _____ is used to convert an infix expression into a postfix expression.

Answer: STACK

2. Underflow takes when _____.

Answer: TOP = -1

3. The order of evaluation of a postfix expression is from _____.

Answer: Left to Right

4. New nodes are added at _____ of the queue.
Answer: Rear

5. _____ allows insertion of elements at either ends but not in the middle.
Answer: Double ended queue (Dequeue)

6. In _____, insertions can be done only at one end, while deletions can be done from both the ends.
Answer: Input restricted dequeue

7. Dequeue is implemented using _____.
Answer: Circular double linked list or Circular array.